WezTerm, Neovim, Herdr 튜토리얼

이 튜토리얼은 이 저장소가 설치하는 터미널 환경을 사용하는 방법을 설명합니다.

터미널 멀티플렉서와 modal editor가 처음인 사람을 기준으로 씁니다.

> 도구 이름은 Herdr입니다. Herder가 아닙니다. 셸 alias h로 시작합니다.

이 튜토리얼은 `./install.sh --apply`를 실행한 뒤 저장소 위치가 그대로 유지된 상태를 전제로 합니다. 실제 설정은 그 clone을 가리키는 Stow link로 구성됩니다. 설치 후 저장소를 이동하거나 이름을 바꾸거나 삭제하면 link가 깨집니다. 적용하기 전에 오래 둘 위치를 먼저 정하세요.

1. 전체 구조 이해하기

세 도구는 역할이 다릅니다.

```
WezTerm 창
 └── Herdr 작업공간(workspace)
 ├── 패널(pane): Claude Code 또는 Codex
 ├── 패널: Neovim
 └── 패널: 테스트, 서버, 일반 셸
```

- WezTerm은 macOS 터미널 창입니다. 텍스트를 그려 주고, 키보드 입력을 받고, 탭을 제공하며, 상단 시스템 상태 표시줄을 보여 줍니다.
- Herdr는 터미널 안에서 지속되는 작업공간, 탭, 패널을 관리합니다. 오래 실행되는 코딩 에이전트를 다룰 때 특히 유용합니다.
- Neovim은 파일을 편집하고, 내가 만든 변경이나 에이전트가 만든 변경을 검토하는 편집기입니다.

이 설정은 WezTerm의 멀티플렉서(multiplexer) 기능을 켜지 않습니다. 에이전트 중심 작업에는 Herdr를 사용하고, 일반 작업이나 원격 작업에는 tmux를 사용하세요. WezTerm, Herdr, tmux를 모두 겹겹이 중첩하는 방식은 피하는 편이 좋습니다.

2. WezTerm에서 시작하기

Spotlight나 Applications 폴더에서 WezTerm을 엽니다. 기본 zsh 셸이 자동으로 시작됩니다.

상단 바의 왼쪽에는 WezTerm 탭이, 오른쪽에는 배터리와 시간이 표시됩니다.

```
〔1. zsh〕 BAT+ 72% | Tue Sep 15 14:30
```

- `BAT+`는 배터리가 충전 중이라는 뜻입니다. `BAT`는 충전 중이 아니라는 뜻입니다.
- 표시는 10초마다 갱신됩니다.

macOS에서 유용한 WezTerm 단축키

동작	단축키
새 WezTerm 탭	`Command-t`
현재 탭 닫기	`Command-w`
이전/다음 탭	`Command-Shift-[` / `Command-Shift-]`
탭 1-9 선택	`Command-1` … `Command-9`
새 WezTerm 창	`Command-n`
이전/다음 창	`Command-\` / Command-Shift-\`
창 1-9 선택	`Command-Option-1` … `Command-Option-9`
활성 패널을 새 창으로 이동	`Command-Shift-m`
한 페이지 스크롤	`Command-Up` / `Command-Down`
세 줄 스크롤	`Command-Shift-Up` / `Command-Shift-Down`
복사/붙여넣기	`Command-c` / `Command-v`
터미널 출력 검색	`Command-f`
글꼴 크게/작게	`Command-+` / `Command--`
글꼴 크기 초기화	`Command-0`
WezTerm 설정 다시 읽기	`Command-r`

창 제목에는 `[1]`, `[2]` 같은 현재 창 번호가 표시됩니다. 이 번호는 `Command-Option-1`부터 `Command-Option-9`까지의 창 선택 단축키와 대응합니다.

WezTerm 탭은 하나의 창 안에 있는 탭입니다.

`Command-Shift-m`은 현재 활성 Herdr 패널 또는 셸 패널을 별도의 창으로 꺼내며, 별도의 대화형 셸 명령을 입력할 필요가 없습니다.

서로 다른 최상위 활동은 WezTerm 탭으로 나누세요. 한 프로젝트 안의 관련 프로세스들은 Herdr의 탭과 패널로 나누는 편이 좋습니다.

Gruvbox 주간·야간 테마 바꾸기

테마 명령은 전역 CLI나 `mat` 명령이 아니라 이 저장소의 스크립트입니다. 저장소로 이동한 뒤 실행합니다. 다른 디렉터리에서는 스크립트의 절대 경로를 사용해도 됩니다.

```
cd /path/to/myagenterminal
./scripts/theme.sh day # 밝은 Gruvbox Soft
./scripts/theme.sh night # 어두운 Gruvbox Soft
./scripts/theme.sh status # 저장된 선택값 확인
```

처음 설치해 선택 파일이 없으면 night가 기본값입니다. 선택값은 Git으로 추적하지 않는 `~/.config/myagenterminal/theme`에 저장됩니다. WezTerm은 설정을 다시 읽고, 새로 여는 Neovim은 같은 day/night와 soft 대비를 사용합니다. 이미 열린 Neovim은 다시 시작해야 합니다. Herdr는 자체적으로 실행 중인 터미널의 밝기를 감지하여 Gruvbox/Light를 고릅니다. 따라서 WezTerm 밖의 터미널에서 Herdr를 열면 저장된 선택값과 화면이 다를 수 있고, Herdr의 강조색도 Soft 팔레트와 완전히 같지는 않습니다. `status`는 실제 모든 창의 색이 아니라 저장된 선택값을 표시합니다.

3. 셸 편의 기능

셸에는 몇 가지 짧은 명령이 정의되어 있습니다.

명령	뜻
`h`	Herdr 시작 또는 연결
`v`	Neovim 시작
`t`	tmux 시작 또는 연결
`ll`	Git 정보가 포함된 자세한 디렉터리 목록
`tree`	디렉터리 트리 보기

`ls`, `ll`, `tree`는 표시한 경로에 터미널 하이퍼링크를 붙입니다. URL은 `Command`를 누른 채 클릭하면 브라우저에서 열리고, `file://` 경로는 macOS의 기본 앱으로 열립니다. Codex, Neovim, 또는 다른 패널 앱이 마우스 입력을 사용하고 있어도 이 기능은 유지됩니다. 설정된 Command modifier는 WezTerm이 처리하기 때문입니다.

그 밖의 대화형 기능은 다음과 같습니다.

- 명령 기록 기반 자동 제안
- 실행 전 잘못된 명령을 알려 주는 구문 색상 표시
- 추가 명령 자동완성
- Atuin 기록 검색
- fzf 흐림 검색(fuzzy selection)
- zoxide 디렉터리 이동
- Starship 프롬프트

희미한 autosuggestion이 보이면 오른쪽 화살표 키를 눌러 받아들입니다.

명령 기록 검색은 `Ctrl-r`을 사용합니다. zoxide가 자주 가는 디렉터리를 학습한 뒤에는 `z project-name`으로 긴 `cd` 명령을 대체할 수 있습니다.

Starship 프롬프트 읽기

Starship은 현재 디렉터리, Git 맥락, 활성 언어 런타임을 한 줄에 표시합니다.

예를 들면 다음과 같습니다.

```
 ridge on git:feat/131-iam-tag-based-model [$!?] via  v3.9.6 via  v24.21.0
```

구간	의미
`ridge`	현재 디렉터리
`on git:feat/131-iam-tag-based-model`	현재 Git 브랜치
`[$✘!?]`	남아 있는 Git 상태: stash 변경사항(`$`), 삭제된 파일(`✘`), 수정된 파일(`!`), 추적되지 않은 파일(`?`)
`via  v3.9.6`	활성 Python 버전
`via  v24.21.0`	활성 Node.js 버전

이 저장소의 프롬프트 순서는 `directory`, `git_branch`, `git_status`, `python`, `nodejs`, `rust`, 명령 실행 시간입니다. 각 구간은 해당 맥락이 있을 때만 표시됩니다. 예를 들어 Git 작업 트리가 깨끗하면 Git 상태 구간은 나타나지 않습니다.

이 설정에 포함된 의도적인 도구들

아래 도구들은 작업 흐름을 돕기 위해 설치됩니다. 임의로 추가된 것이 아닙니다.

`./install.sh --apply --packages`는 `Brewfile`의 Homebrew 항목을 설치합니다.

Claude Code와 Codex는 예외입니다. 이 둘은 배포자가 제공하는 네이티브

설치 프로그램을 사용하므로 `./scripts/agents.sh`로 따로 설치합니다.

도구	용도	시작점
Atuin	셸 기록 검색과 재사용	↑ 또는 `Ctrl-r`; 검색어 입력, `Enter`는 선택한 명령을 편집, `Esc`는 종료
fzf	셸 통합에서 쓰는 흐림 검색	`Ctrl-t` 파일, `Alt-c` 디렉터리, 또는 선택기 명령에서 사용
zoxide	자주 가는 디렉터리로 이동	`z project-name`
Starship	Git과 실행 환경 맥락을 보여 주는 프롬프트	zsh 시작 시 자동 실행
zsh plugins	제안, 구문 강조, 자동완성	평소처럼 입력하고 `Right Arrow`로 제안 수락
bat	구문 강조가 있는 파일 읽기	`cat README.md`
eza	현대적인 목록과 트리 출력	`ls`, `ll`, `tree`; 출력된 경로를 ⌘-클릭
ripgrep / fd	빠른 텍스트 검색 / 파일 찾기	`rg "pattern" .` / `fd filename`
jq	JSON 검사와 변환	`jq '.items[]' data.json`
delta	읽기 쉬운 Git diff	`git diff`
mise	Node 24와 pnpm 10.28.0 관리	`mise current`, `mise install`
uv	Python 프로젝트 환경 관리	Python 프로젝트 안에서 `uv sync` 또는 `uv run ...`
GitHub CLI	터미널에서 GitHub 작업	`gh auth status`, `gh pr list`

Atuin의 위쪽 화살표 화면은 의도된 동작입니다. 일반적인 한 줄씩의 기록 이동을 검색 가능한 기록 목록으로 대체합니다. ↑/↓로 결과를 고르고, `Enter` 또는 `Tab`을 누르면 선택한 명령이 프롬프트에 편집 가능한 상태로 들어갑니다. `Ctrl-o`는 내용을 살펴보고, `Esc`는 아무것도 선택하지 않고 나갑니다. 명령은 일반 셸 프롬프트에서 다시 제출하기 전까지 실행되지 않습니다.

4. 먼저 Neovim의 모드를 익히기

프로젝트에서 Neovim을 시작합니다.

```
cd /path/to/project
v .
```

Neovim은 모달 편집기(modal editor)입니다. 같은 키라도 현재 모드에 따라 다른 일을 합니다.

모드	용도	들어가는 법	나가는 법
일반 모드(Normal)	키 명령으로 이동하고 편집	Neovim은 여기서 시작	-
입력 모드(Insert)	실제 텍스트 입력	`i`, `a`, `o`	`Esc`
선택 모드(Visual)	텍스트 선택	`v` 또는 `V`	`Esc`
명령줄(Command-line)	저장, 종료, 편집기 명령 실행	`:`	`Enter` 또는 `Esc`

어느 모드인지 헷갈리면 `Esc`를 누르세요. 대부분의 상황에서 일반 모드로 돌아옵니다.

첫 편집

1. `Space Space`를 눌러 똑똑한 파일 선택기(smart file picker)를 엽니다.
2. 파일 이름의 일부를 입력합니다.
3. 화살표 키로 선택하고 `Enter`를 누릅니다.
4. 처음에는 `h`, `j`, `k`, `l` 또는 화살표 키로 이동합니다.
5. `i`를 누르고 내용을 수정한 뒤 `Esc`를 누릅니다.

6. `Space w`로 저장합니다.
7. `:q`를 입력하고 `Enter`를 눌러 종료합니다.

자주 쓰는 종료 명령은 다음과 같습니다.

명령	뜻
`:w`	저장
`:q`	저장하지 않은 변경이 없을 때 종료
`:wq`	저장하고 종료
`:q!`	저장하지 않은 변경을 버리고 종료

기본 일반 모드 이동과 편집

동작	키
왼쪽/아래/위/오른쪽 이동	`h` / `j` / `k` / `l`
다음/이전 단어	`w` / `b`
줄 시작/끝	`0` / `$`
파일 시작/끝	`gg` / `G`
반 페이지 위/아래	`Ctrl-u` / `Ctrl-d`
실행 취소/다시 실행	`u` / `Ctrl-r`
커서 아래 문자 삭제	`x`
현재 줄 삭제	`dd`
현재 줄 복사	`yy`
커서 뒤에 붙여넣기	`p`
파일 안에서 검색	`/text`, 그다음 `Enter`
다음/이전 검색 결과	`n` / `N`

시스템 클립보드가 켜져 있습니다. Neovim에서 복사한 텍스트를 다른 macOS 앱에 붙여넣을 수 있고, 반대로 macOS 앱에서 복사한 텍스트를 Neovim에 붙여넣을 수도 있습니다.

왜 `q`와 `:q`는 다른 일을 하는가

일반 모드는 Neovim의 기본 키 명령(command-key) 상태입니다. 이 상태에서 `j`를 누르면 파일에 글자 `j`를 입력하는 대신 커서가 아래로 움직입니다. 글자를 입력하고 싶을 때는 `i`로 입력 모드에 들어가고, 다시 `Esc`로 일반 모드로 돌아옵니다. 일반 모드에서 `:`를 누르고, 아래에 열린 명령줄에 `q`를 입력한 뒤 `Enter`를 누르면 현재 창을 닫습니다. 마지막 창이었다면 Neovim도 종료됩니다. 이 전체 입력을 `:q`라고 적습니다.

반대로 일반 파일의 일반 모드에서 `q`를 바로 누르면 **매크로 기록(macro recording)** 명령이 시작되고, `a` 같은 레지스터(register) 이름을 기다립니다. 대기 중인 `q`가 화면 아래 상태 영역에 보일 수 있지만 명령줄을 연 것은 아닙니다. 실수였다면 `Esc`를 눌러 취소하세요. 기록 중에는 `q`를 다시 누르면 기록이 끝납니다. 특수 플러그인 화면에서는 같은 키가 다른 뜻을 가질 수 있습니다. 예를 들어 Neogit 상태 화면에서 `q`는 Neogit을 닫습니다.

매크로는 Neovim 키 입력의 순서를 기록했다가 다시 실행하는 기능입니다. 세 줄 이상 있는 파일에서 커서만 움직이는 예시를 해 보세요. 이 예시는 파일을 수정하지 않습니다.

1. gg를 눌러 첫 줄로 이동합니다.
2. q, 그다음 a를 눌러 레지스터 a에 기록을 시작합니다.
3. j를 눌러 한 줄 내려간 뒤 q를 눌러 기록을 멈춥니다.
4. @a를 눌러 매크로를 재생합니다. 커서가 한 줄 더 내려갑니다.

편집 매크로는 재생할 때마다 실제 텍스트를 바꿀 수 있으니, 많이 반복하기 전에 무엇을 기록했는지 확인하세요. Herdr의 `Ctrl-b`, 그다음 `[` 복사 모드는 다른 기능입니다. 그것은 터미널 출력의 스크롤백(scrollback)을 이동하고 복사하기 위한 기능이며, 파일을 편집하거나 Neovim 키 입력을 기록하지 않습니다. 이 복사 모드의 `q`는 복사 모드를 나가는 키입니다.

5. 이 Neovim 설정 사용하기

아래 표에서 `Space`는 리더 키(leader key)입니다. 예를 들어 `Space g g`는 Space를 누르고 손을 뗀 뒤 `g`를 두 번 누르라는 뜻입니다.

코드 찾기

동작	단축키
똑똑한 파일 선택기	`Space Space`
프로젝트 전체 텍스트 검색	`Space /`
Git branch 탐색	`Space g b`
Git 기록 탐색	`Space g l`

선택기는 흐림 검색(fuzzy search)을 지원합니다. 파일 이름이나 문구의 특징적인 일부만 입력하고 결과를 선택한 뒤 `Enter`를 누릅니다. 닫으려면 `Esc`를 누릅니다.

Oil로 폴더 탐색, Snacks로 빠른 검색

Oil은 현재 파일의 상위 폴더를 Neovim 버퍼처럼 보여 주는 파일 탐색기입니다. `Space e` 또는 `:Oil`로 열고, `Enter`로 파일·폴더를 열거나 `-`로 상위 폴더로 이동합니다. `g?`는 Oil 화면의 도움말입니다. 숨김 파일도 보입니다. Oil에서 파일 이름을 고치거나 행을 추가·삭제하면 실제 파일 작업이 될 수 있습니다. `:w`로 저장할 때 적용되므로 변경 내용을 확인하세요.

Snacks는 별도의 빠른 선택기(picker)와 UI 기능 묶음입니다. `Space Space`는 파일 찾기, `Space /`는 프로젝트 텍스트 검색, `Space g b`와 `Space g l`은 각각 Git branch와 기록을 찾습니다. 이 저장소는 Snacks의 입력창·알림, 큰 파일 처리와 빠른 파일 열기도 켭니다. Oil은 폴더를 따라가며 파일을 관리할 때, Snacks 선택기는 이름이나 내용을 알고 빠르게 찾을 때 쓰세요.

터미널에서 Markdown 미리보기

Markdown 파일을 연 상태에서 `Space m p`를 누르거나 `:RenderMarkdown preview`를 실행하면 렌더링된 미리보기가 오른쪽에 열립니다. 미리보기는 Neovim 안에서 동작하며, 원문은 Markdown 문법 그대로 둔 채 저장 전 수정도 반영합니다. `Ctrl-h`와 `Ctrl-l`로 두 창 사이를 이동하고, 원문 창에서 `Space m p`를 다시 누르면 미리보기가 닫힙니다.

처음 Neovim을 실행하면 lazy.nvim이 미리보기 플러그인을 설치합니다.

WezTerm에서 오른쪽 미리보기를 스크롤할 때 왼쪽 원문 화면 일부가 지워져 보이면, Neovim 명령줄에서 `:redraw!`를 실행해 화면을 다시 그립니다.

`Ctrl-l`은 이 설정에서 오른쪽 창 이동 키이므로 새로고침에 사용할 수 없습니다.

파일 내용이 바뀐 것은 아닙니다. 이 현상은 구형 WezTerm에서 Neovim의
세로 분할 스크롤과 충돌하는 〔알려진 문제〕(https://github.com/neovim/neovim/issues/35133)입니다.
지속적인 해결 방법은 〔WezTerm의 안내〕(https://wezterm.org/config/lua/config/term.html)에
따라 `wezterm` terminfo를 설치한 뒤 WezTerm 설정에 `config.term = "wezterm"`을
추가하는 것입니다. 새 터미널 탭을 열어 `echo $TERM`이 `wezterm`인지 확인하세요.

편집기 분할 다루기

`:vsplit` 또는 `:split`으로 분할을 만든 뒤 다음 키로 이동합니다.

방향	단축키
왼쪽	`Ctrl-h`
아래	`Ctrl-j`
위	`Ctrl-k`
오른쪽	`Ctrl-l`

이것은 하나의 터미널 패널 안에 있는 Neovim 분할입니다. Herdr 패널과는
다릅니다.

Gitsigns로 Git 변경 검토하기

변경된 줄은 왼쪽 열에 표시됩니다. 현재 줄에는 잠시 뒤 Git blame도
표시됩니다.

동작	단축키
다음/이전 변경 hunk	`]h` / `[h`
현재 hunk 미리보기	`Space h p`
현재 hunk stage	`Space h s`
현재 hunk reset	`Space h r`
현재 줄 blame 켜기/끄기	`Space h b`

`Space h r`은 조심하세요. 현재 uncommitted hunk를 버립니다.

Neogit으로 저장소 전체 검토하기

`Space g g`를 누르면 Neogit이 열립니다. Neogit은 파일, diff, staging,
commit을 저장소 전체 관점에서 검토하는 Git 화면입니다. Neogit 안에서 `?`를
누르면 현재 상황에 맞는 조작법이 나옵니다. 처음부터 전부 외우려 하지 말고
화면에 표시되는 조작법을 우선 사용하세요.
상태 화면, section, hunk, @@ diff header를 기초부터 이해하려면
〔한국어 Neogit 가이드〕(neogit-guide.ko.md)를 읽으세요.
Space를 누른 뒤 잠시 멈추면 which-key도 사용 가능한 리더 키 동작을
보여 줍니다.

6. Herdr 작업공간 시작하기

실제 Git 프로젝트에서 시작합니다.

```
cd /path/to/project
h
```

`h`는 `herdr`와 같습니다. 기본 지속 Herdr 세션을 시작하거나 연결하고,
작업공간을 만들거나 엽니다.
Herdr의 계층은 다음 순서로 이해하세요.

```
세션(session)
 └── 작업공간(workspace): 하나의 프로젝트나 작업
 └── 탭(tab): "development" 같은 하나의 배치(layout)
 └── 패널(pane): 하나의 셸, 편집기, 에이전트, 테스트 프로세스, 서버
```

- 세션(session)은 백그라운드 Herdr 서버입니다. 대부분의 사람은 기본 세션

하나면 충분합니다.

- 작업공간(workspace)은 보통 하나의 저장소나 작업을 나타냅니다.
- 탭(tab)은 `agents`, `tests`, `server` 같은 패널 배치를 묶습니다.
- 패널(pane)은 실제 터미널 프로세스 하나를 담습니다.

처음에는 마우스로 시작해도 괜찮습니다. 패널이나 탭을 클릭해 초점을 옮기고,

패널 경계를 드래그해 크기를 조절하고, 우클릭으로 상황별 메뉴를 엽니다.

7. Herdr prefix 이해하기

Herdr 명령은 `Ctrl-b`를 prefix로 사용합니다. 이것은 한 번에 누르는 큰

키 조합이 아니라 순서입니다.

`Ctrl-b`, 그다음 `v`는 다음과 같이 누릅니다.

1. Ctrl을 누른 채 `b`를 누릅니다.
2. 둘 다 손을 뗍니다.
3. `v`를 누릅니다.

첫 단계는 초점이 있는 셸이나 편집기에 입력을 보내는 대신, 다음 키를 Herdr

명령으로 해석하라고 Herdr에게 알려 줍니다.

언제든 `Ctrl-b`, 그다음 `?`를 누르면 현재 키 바인딩을 볼 수 있습니다.

긴 Codex 출력을 Neovim에서 읽기(마우스 불필요)

초점이 있는 Codex 패널의 Herdr 스크롤백을 바로 Neovim에서 열 수 있습니다.

새 패널을 만들거나 복사 모드에 먼저 들어갈 필요가 없습니다.

1. Codex 패널에 초점을 둡니다. 필요하면 `Ctrl-b`, 그다음 `h/j/k/l`로

이동합니다.

2. `Ctrl-b`를 누르고 둘 다 손을 뗀 뒤 `e`를 누릅니다.
3. Herdr가 그 패널의 스크롤백을 `$EDITOR`로 엽니다. 이 설정은

`EDITOR=nvim`이므로 Neovim이 자동으로 열립니다.

4. `Esc`로 일반 모드를 확인합니다. 처음/끝은 `gg` / `G`, 반 페이지

이동은 `Ctrl-u` / `Ctrl-d`, 검색은 `/keyword`, `Enter`, 그다음

`n` / `N`을 사용합니다. 한 줄씩 조정하려면 `j` / `k`를 사용합니다.

5. `:q`를 입력하고 `Enter`를 눌러 편집기를 닫고 Herdr로 돌아옵니다.

이 방식은 Herdr가 보관한 패널 텍스트를 엽니다. 반드시 Codex 대화 전체를

여는 것은 아닙니다. 전체 화면 앱이 자기 내부 기록에만 오래된

답변을 보관해서 보이지 않는다면, Codex에게 그 답변을 Markdown 파일로 저장해

달라고 한 뒤 셸 패널에서 `v path/to/file.md`로 파일을 여세요.

필수 Herdr 키

동작	순서
도움말 보기	`Ctrl-b`, 그다음 `?`
오른쪽에 새 패널로 분할	`Ctrl-b`, 그다음 `v`
아래에 새 패널로 분할	`Ctrl-b`, 그다음 `-`
왼쪽/아래/위/오른쪽 패널에 초점 이동	`Ctrl-b`, 그다음 `h/j/k/l`
다음 패널로 순환	`Ctrl-b`, 그다음 `Tab`
패널 닫기	`Ctrl-b`, 그다음 `x`
패널 확대/복원	`Ctrl-b`, 그다음 `z`
크기 조절 모드 들어가기	`Ctrl-b`, 그다음 `r`
패널 스크롤백을 복사 모드로 보기	`Ctrl-b`, 그다음 `[`
패널 스크롤백을 `$EDITOR`로 열기	`Ctrl-b`, 그다음 `e`
사이드바 켜기/끄기	`Ctrl-b`, 그다음 `b`
Herdr에서 분리(detach)	`Ctrl-b`, 그다음 `q`

탭과 작업공간

동작	순서
새 탭	`Ctrl-b`, 그다음 `c`
이전/다음 탭	`Ctrl-b`, 그다음 `p/n`
탭 1-9 선택	`Ctrl-b`, 그다음 `1` … `9`
탭 이름 변경	`Ctrl-b`, 그다음 `Shift-t`
탭 닫기	`Ctrl-b`, 그다음 `Shift-x`
작업공간 선택기 열기	`Ctrl-b`, 그다음 `w`
작업공간 만들기	`Ctrl-b`, 그다음 `Shift-n`
작업공간 이름 변경	`Ctrl-b`, 그다음 `Shift-w`
작업공간 닫기	`Ctrl-b`, 그다음 `Shift-d`
Git worktree 작업공간 만들기	`Ctrl-b`, 그다음 `Shift-g`

대문자 동작은 prefix에서 손을 뗀 뒤 동작 키를 Shift와 함께 누르라는 뜻입니다.

마우스나 Page Up/Down 키 없이 스크롤백 보기

Herdr의 전체 화면 인터페이스가 활성화되어 있으면 Herdr가 자기 패널의 스크롤백을 소유합니다. 그래서 WezTerm의 `ScrollByPage` 단축키는 Herdr 패널 기록을 이동하지 않습니다. 빠르게 훑어보려면 Herdr 복사 모드에 들어갑니다.

```
Ctrl-b, 그다음 [
```

복사 모드 안에서는 다음 키를 사용합니다.

동작	키
반 페이지 위/아래 이동	`Ctrl-u` / `Ctrl-d`
한 줄 위/아래 이동	`k` / `j`
이전/다음 문단으로 이동	`{` / `}`
앞/뒤 검색	`/` / `?`
검색을 앞/뒤로 반복	`n` / `N`

동작	키
복사 모드 나가기	`q` 또는 `Esc`

복사 모드 커서는 현재 출력의 맨 아래에서 시작합니다. `Ctrl-u`나 `Ctrl-d`로 크게 이동한 뒤에는 `k`와 `j`로 그 커서를 한 줄씩 조정합니다. 검색이나 정밀한 위치 이동이 필요하면 복사 모드를 나와 위의 Codex-to-Neovim 절차(`Ctrl-b`, 그다음 `e`)를 사용하세요.

`Ctrl-b`, 그다음 `Shift-d`로 작업공간을 닫으면 확인 후 그 작업공간의 Herdr 패널들이 닫힙니다. 프로젝트 디렉터리나 연결된 Git branch 또는 worktree는 삭제하지 않습니다.

8. 실제 에이전트 작업 흐름

프로젝트를 열고 Herdr를 시작합니다.

```
cd ~/Documents/Projects/example-project
h
```

첫 패널은 에이전트에 사용합니다.

```
claude
```

또는:

```
codex
```

그다음 이런 배치를 만듭니다.

1. `Ctrl-b`, 그다음 `v`를 눌러 오른쪽에 패널을 만듭니다.
2. 새 패널에서 `v .`를 실행해 Neovim을 엽니다.
3. `Ctrl-b`, 그다음 `-`를 눌러 아래에 패널을 만듭니다.
4. 아래 패널에서 프로젝트의 테스트나 개발 서버를 실행합니다.

결과는 다음과 같습니다.

```
┌──────────────────────────────┬──────────────────────────────┐
│ Claude Code / Codex          │ Neovim                       │
│                              │ 검토와 편집                  │
├──────────────────────────────┴──────────────────────────────┤
│ 테스트, 서버, 로그, 또는 셸                                 │
└─────────────────────────────────────────────────────────────┘
```

에이전트가 파일을 변경하면:

1. `Ctrl-b`, 그다음 `l` 또는 클릭으로 Neovim에 초점을 둡니다.
2. `]h`와 `[h`로 변경 hunk를 방문합니다.
3. `Space h p`로 hunk를 살펴봅니다.
4. `Space g g`로 저장소 전체를 검토합니다.
5. 아래 패널에서 관련 테스트를 실행합니다.
6. 에이전트 패널로 돌아가 피드백이나 다음 작업을 줍니다.

9. 분리, 다시 연결, 중지

패널을 멈추지 않고 Herdr에서 빠져나갑니다.

```
Ctrl-b, 그다음 q
```

WezTerm 창을 닫아도 됩니다. Herdr 백그라운드 서버와 패널 프로세스는

계속 실행됩니다.

나중에 WezTerm을 열고 다시 연결합니다.

```
h
```

Herdr 인터페이스 밖에서 상태를 확인하려면:

```
herdr status
herdr workspace list
herdr session list
```

기본 Herdr 서버와 실행 중인 환경을 실제로 멈추려면:

```
herdr server stop
```

단순히 인터페이스에서 나가려고 `server stop`을 사용하지 마세요. 그때는 분리(detach)를 사용하세요.

10. 에이전트 완료 알림

Herdr는 에이전트 상태를 아는 알림 출처이고, WezTerm은 macOS 알림을 전달합니다. 설정된 흐름은 다음과 같습니다.

```
Claude Code 또는 Codex 상태 변경
↓
Herdr가 완료 또는 주의 필요 상태 인식
↓
Herdr가 터미널 알림 발생
↓
WezTerm이 macOS에 표시 요청
```

같은 WezTerm 창에 초점이 있어도 알림은 전달됩니다. 에이전트 완료 또는 주의 필요 이벤트가 조용히 묻히지 않도록 하기 위해서입니다. 백그라운드 작업공간의 에이전트 변경에 대해서도 Herdr 소리가 켜져 있습니다.

WezTerm은 처음에 macOS 임시 알림 권한(provisional notification authorization)을 요청할 수 있습니다. macOS는 임시 권한 요청에 대해 권한 대화상자를 보여 주지 않으며, 첫 알림을 배너 대신 알림 센터(Notification Center)에 조용히 넣을 수 있습니다. 전체 경로를 테스트한 뒤 배너가 보이지 않으면 알림 센터를 확인하세요.

```
herdr notification show "myagenterminal" --body "Notifications are working" --sound done
```

이 저장소는 WezTerm에 초점이 있을 때도 알림을 보이도록 다음 설정을 켭니다.

```
config.notification_handling = "AlwaysShow"
```

이 설정을 수동으로 바꾼 뒤에는 `Command-r`로 WezTerm을 다시 읽으세요.

이 경로는 모든 터미널 벨에 반응하는 것보다 정확합니다. Herdr는 지원되는 에이전트의 상태 변화를 알지만, WezTerm만으로는 어떤 프로그램이 알림 또는 벨 시퀀스를 보냈는지만 알 수 있기 때문입니다.

11. 선택적 에이전트 통합

Herdr는 터미널 출력에서 지원되는 에이전트를 감지할 수 있습니다. 선택적인 네이티브 통합(native integration)은 더 풍부한 생명주기 상태나 세션 복원을

제공할 수 있습니다.

현재 상태를 확인합니다.

```
herdr integration status
```

원하는 통합만 설치합니다.

```
herdr integration install claude
herdr integration install codex
```

이 통합들은 `AGENTS.md`와 별개입니다. 다만 에이전트의 설정 디렉터리 안에 hook 파일을 작성합니다. 자동 myagenterminal 설치의 일부가 아니라, 명시적으로 선택하는 선택적 설정 단계로 취급하세요.

12. 문제 해결

Herdr 단축키가 아무 일도 하지 않는다

`Ctrl-b`를 누르고 손을 뗀 뒤 동작 키를 누르세요. `Ctrl-b`, 그다음 `?`로 현재 바인딩을 확인합니다. macOS 데스크톱이나 바깥 터미널이 일부 직접 키 조합을 가로챌 수 있지만, 설정된 prefix sequence는 대부분의 충돌을 피합니다.

실수로 Herdr 안에서 Herdr를 다시 열었다

중첩 Herdr 세션은 비활성화되어 있습니다. `$HERDR_ENV`가 `1`이면 이미 Herdr 패널 안에 있는 것입니다.

```
echo $HERDR_ENV
```

`h`를 다시 실행하지 말고 기존 작업공간을 사용하세요.

에이전트가 감지되지 않는다

다음을 실행합니다.

```
herdr agent list
herdr integration status
```

Herdr 로그는 `~/.config/herdr/` 아래에 있습니다. 이 디렉터리는 myagenterminal 저장소로 link되어 있으므로, 해당 `*.log` 파일과 named-session 실행 디렉터리는 Git에서 명시적으로 무시됩니다. `config.toml`만 추적 대상이어야 합니다.

에이전트 완료 알림이 나타나지 않는다

macOS System Settings에서 WezTerm 알림이 허용되어 있는지 확인합니다. `Command-r`로 WezTerm 설정을 다시 읽고, `herdr server reload-config`로 Herdr를 다시 읽은 뒤, 10장의 알림 테스트를 실행합니다. 설정된 `AlwaysShow` 모드는 WezTerm 창에 초점이 있어도 알림을 전달합니다. 처음 사용 시 임시 권한 때문에 알림이 배너 대신 알림 센터에 조용히 들어갈 수 있습니다.

Neovim이 멈춘 것처럼 보인다

`Esc`를 누른 뒤 저장할지 결정합니다.

```
:wq 저장하고 종료
:q! 변경을 버리고 종료
```

WezTerm 상태 표시줄이 갱신되지 않는다

`Command-r`로 설정을 다시 읽습니다. 그래도 보이지 않으면 WezTerm을 재시작하고 다음 명령으로 설정을 검증합니다.

```
wezterm --config-file ~/.config/wezterm/wezterm.lua ls-fonts >/dev/null
```

URL이나 로컬 파일이 열리지 않으면 그 경로가 `ll` 또는 `tree`가 출력한 경로인지 확인하고, `Command`를 누른 채 클릭한 뒤, `Command-r`로 WezTerm을 다시 읽으세요. `file://` 경로는 macOS가 어떤 기본 앱으로 열지 결정합니다.

13. 작은 연습

버려도 되는 Git 저장소를 사용합니다.

```
mkdir -p /tmp/myagenterminal-practice
cd /tmp/myagenterminal-practice
git init
printf '# Practice\n' > README.md
h
```

그다음:

1. `v .`로 Neovim을 엽니다.
2. `Space Space`로 `README.md`를 찾습니다.
3. 새 줄을 추가하고 `Space w`로 저장합니다.
4. `Space h p`로 변경 hunk를 미리 봅니다.
5. `Space g g`로 Neogit을 엽니다.
6. `Ctrl-b`, 그다음 `v`로 Herdr 분할을 만듭니다.
7. 새 패널에서 `git diff`를 실행합니다.
8. `Ctrl-b`, 그다음 `q`로 분리합니다.
9. `h`로 다시 연결하고 패널이 살아 있는지 확인합니다.

이 순서가 익숙해지면 같은 배치를 실제 프로젝트에서 사용하세요.

공식 참고 문서

- WezTerm: 〈https://wezterm.org/config/files.html〉
- Neovim 사용자 설명서: 〈https://neovim.io/doc/user/〉
- Neovim 매크로 기록과 재생: 〈https://neovim.io/doc/user/usr_10/〉
- Oil 파일 탐색기: 〈https://github.com/stevearc/oil.nvim〉
- Snacks 선택기와 UI 기능: 〈https://github.com/folke/snacks.nvim〉
- Neovim Markdown 미리보기: 〈https://github.com/MeanderingProgrammer/render-markdown.nvim〉
- Herdr 개념: 〈https://herdr.dev/docs/concepts/〉
- Herdr 키보드 가이드: 〈https://herdr.dev/docs/keyboard/〉
- Herdr 설정: 〈https://herdr.dev/docs/configuration/〉